

PROBLEM 4

Git Collaboration for a Social Media App

UNIT – III: Full-Stack Development, Containerization, Version Control, and Advanced Algorithms

1. Recommended Git/GitHub Workflow

The team should use a protected main branch, a shared develop branch, and short-lived feature branches. Developers must never push directly to main. All changes should be developed in feature branches, reviewed through Pull Requests (PRs), validated by Continuous Integration (CI), and merged into develop only after approval. Stable releases are promoted from develop to main through a PR.

2. Git Workflow Diagram

[illegible]

Flow: feature branch → Pull Request → automated CI + code review → develop → release Pull Request → main.

3. Branching Rules

- **main**: Production-ready code only. Direct pushes are prohibited.
- **develop**: Integration branch for completed and reviewed features.
- **feature/<feature-name>**: Individual developer work, created from the latest develop branch.
- **fix/<issue-name>**: Used for bug fixes that are not release-critical.
- **hotfix/<issue-name>**: Used for urgent production fixes and merged back into both main and develop.
- Branches should be short-lived and deleted after successful merging.

4. Pull Request and Approval Rules

- Every feature must be submitted through a Pull Request; direct pushes to main and develop are blocked.
- At least 2 reviewers should approve important or high-risk changes; for normal changes, at least 1 reviewer is required.
- The PR author cannot approve their own PR.
- All review comments marked as changes requested must be resolved before merging.
- The PR description must contain the purpose, changes made, testing performed, and related issue number.
- Branch protection should require approval and successful CI checks before merge.
- Use Squash and Merge for small feature branches to keep the history clean.

5. CI Checks Required Before Merge

CI Check	Purpose	Merge Rule
Linting	Detects coding-style and common static-analysis errors.	Must pass
Unit Tests	Verifies individual functions/components.	Must pass
Build Check	Confirms the application can be compiled/built.	Must pass
Security/Dependency Scan	Detects known vulnerable dependencies.	Must pass or be reviewed
Test Coverage	Checks whether important code paths are tested.	Meet agreed threshold

6. Guidelines for Resolving Merge Conflicts

1. Update the feature branch with the latest develop changes before requesting final review.
2. Identify the conflicting files using git status.
3. Open the files and decide which changes should remain; do not blindly choose one side.
4. Remove conflict markers and run the application plus all unit tests.
5. Commit the resolved changes and push the updated feature branch.
6. Ask the reviewer to re-check the affected logic when the conflict changed behavior.
7. Never force-push shared branches such as main or develop.

7. Example Git Commands

```
git checkout develop<br>git pull origin develop<br>git checkout -b
feature/post-module<br><br>git add .<br>git commit -m "Add post module"<br>git push -u origin
feature/post-module<br><br># Create Pull Request: feature/post-module → develop<br># CI runs
automatically<br># Reviewer approves<br># Merge only when approvals + CI checks pass</pre>

```

8. Implementation Screenshot

The following screenshot is an illustrative GitHub-style implementation view showing a social-media-app repository, the develop branch, Pull Requests, CI workflow files, and project structure.

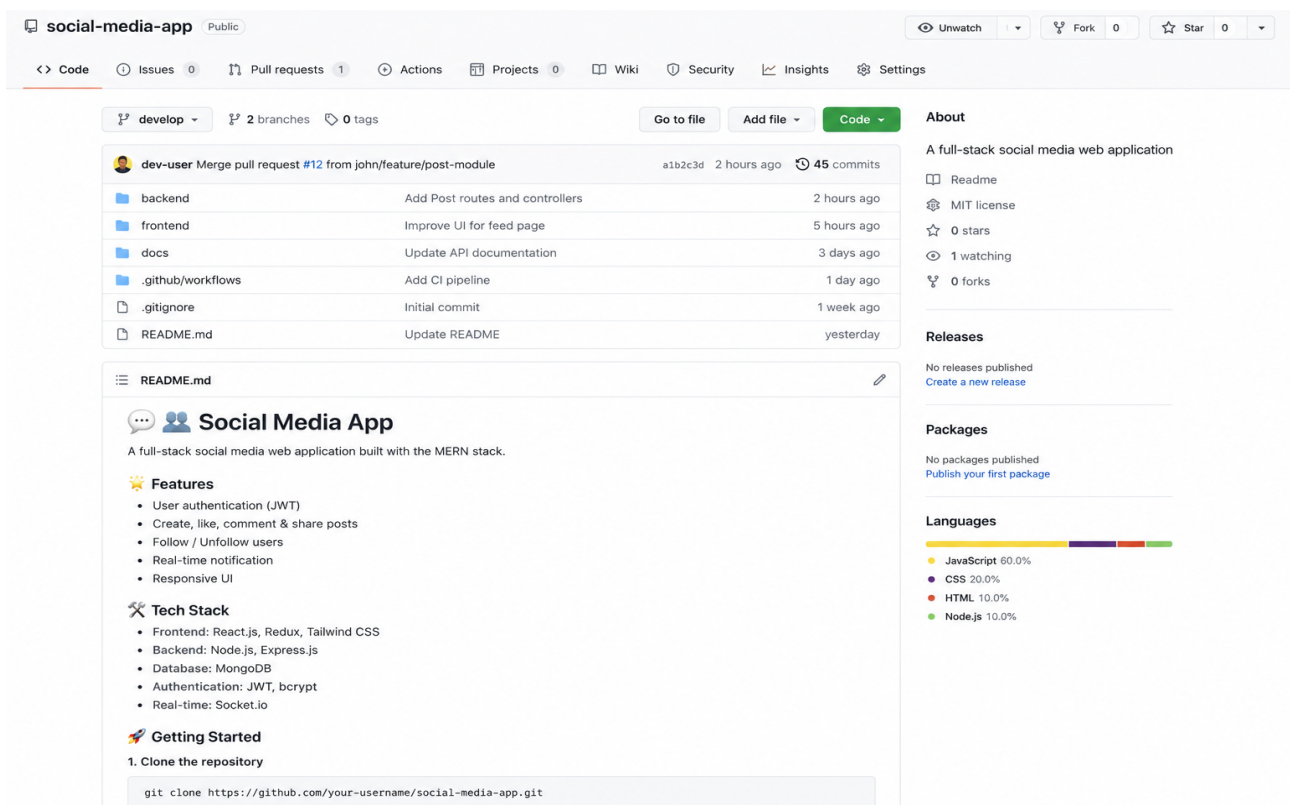


Figure 1: GitHub-style implementation view of the social-media-app repository.

9. Final Workflow Summary

Developer creates a feature branch → implements the feature → pushes the branch → opens a PR to develop → CI performs linting, unit tests, build and security checks → reviewer(s) inspect the code → issues are resolved → PR is merged into develop → release PR is created from develop to main → final CI and approval → main is updated.